

Social Media Platform - Interview Guide

Design: Facebook / Instagram / Twitter Feed Generation

For 12+ Year Experienced Engineers

Quick Navigation

1. [Interview Format](#)
 2. [HLD Interview \(45-60 min\)](#)
 3. [LLD Interview \(45-60 min\)](#)
 4. [Common Questions & Answers](#)
 5. [Cheat Sheet](#)
-

Interview Format

HLD (High-Level Design)

Focus: Feed generation, fan-out strategies, social graph, media pipeline **Time:** 45-60 minutes **Unique challenge:** The celebrity problem — how do you handle Cristiano Ronaldo's 200M followers?

LLD (Low-Level Design)

Focus: Feed ranking, social graph traversal, notification pipeline, concurrency **Time:** 45-60 minutes **Key distinction:** Pull vs Push vs Hybrid fan-out — know when to use each

HLD Interview (45-60 min)

Phase 1: Requirements Gathering (5-10 min)

Interviewer: "Design a social media platform like Instagram."

Your Response — *Say this out loud, confidently:*

"Instagram-scale is a great scope. Let me ask a few questions before I dive in — because the feed generation strategy depends heavily on the follow-graph characteristics."

Functional Requirements — Ask these:

Q: Can users create posts (text + images + video)?

A: Yes — images and short videos, like Instagram.

Q: Can users follow / unfollow other users?

A: Yes — asymmetric follow (Twitter-style, not mutual friends like Facebook).

Q: Does a user see a feed of posts from people they follow?

A: Yes – the home feed is the core feature.

Q: Is the feed chronological or ranked (algorithmic)?

A: Let's say ranked – more realistic and interesting to design.

Q: Do we need likes, comments, shares?

A: Yes – basic engagement features.

Q: Do we need push notifications?

A: Yes – notify followers when someone they follow posts.

Q: Do we need search (hashtags, people)?

A: Mention as extension – let's not deep-dive on it.

Q: Stories / Reels / live streaming?

A: Out of scope – focus on feed posts.

Non-Functional Requirements – Ask these:

Q: Scale – how many users and posts?

A: 500M DAU, 2 billion total users, 100M posts per day.

Q: Read/write ratio?

A: Very read-heavy – 100:1 read:write ratio.

Q: Feed load time requirement?

A: < 200ms to render the feed.

Q: Eventual consistency acceptable for feed?

A: Yes – slight delay (< 5 seconds) is fine.

(If Taylor Swift posts, her followers can see it 2–3 seconds later.)

Q: Availability vs Consistency?

A: High availability – eventual consistency for feed.

Writes (creating a post) can be eventually consistent.

The feed itself can lag slightly.

Write this on the whiteboard:

INSTAGRAM-LIKE REQUIREMENTS SUMMARY	
Functional	Non-Functional
Create posts (image/video)	500M DAU, 2B total users
Follow / unfollow users	100M posts per day
Home feed (ranked)	Read:Write ratio = 100:1
Likes, comments	Feed load < 200ms
Push notifications	Eventual consistency OK (< 5 sec)

[Extension] Search	High availability
--------------------	-------------------

Phase 2: Capacity Estimation (5 min)

"Let me size this to understand what we're building for."

Users:
2 billion registered, 500M DAU

Posts:
100M posts/day = 1,160 posts/sec

Feed reads:
100:1 ratio → 100 × 100M = 10 billion feed reads/day
= 115,000 feed reads/sec (peak ~3× = 345,000/sec)

Follow graph:
Avg user follows 500 people
2B users × 500 = 1 trillion follow relationships
Storage: 1T × 16 bytes = 16 TB (follow pairs)

Media storage:
Avg image: 200 KB
Avg video: 5 MB
100M posts/day × 70% images × 200KB = 14 TB/day (images)
100M posts/day × 30% videos × 5MB = 150 TB/day (videos)
→ ~165 TB/day of new media
→ CDN + tiered storage (hot → warm → cold)

Visual Breakdown:

CAPACITY AT A GLANCE	
Post writes	~1,160/sec (small – write path is easy)
Feed reads	~115,000/sec (HUGE – this is the hard problem)
Media ingested	~165 TB/day (CDN + tiered S3 storage)
Follow graph	~16 TB (Cassandra or graph DB)
Celebrity posts	1 post × 200M followers = 200M fan-out writes ← THIS is the design-defining constraint
Key insight: reads dwarf writes 100:1 → optimize for reads. Key challenge: celebrity posts blow up the write fan-out.	

Phase 3: Core Entities (3 min)

CORE ENTITIES	
User	Account, profile, follower count
Post	Content (text + media URLs), metadata
Follow	Directed edge: follower → followee
Feed	Ordered list of postIds for a user
Like	User → Post association + count
Comment	Text attached to a post
Notification	Event (like, comment, new follower, new post)

Relationships:

User — creates —> Post
User — follows —> User
User — has a —> Feed (pre-computed or on-demand)
Post — has many —> Likes (counter + user list)
Post — has many —> Comments

Phase 4: API Design (5 min)

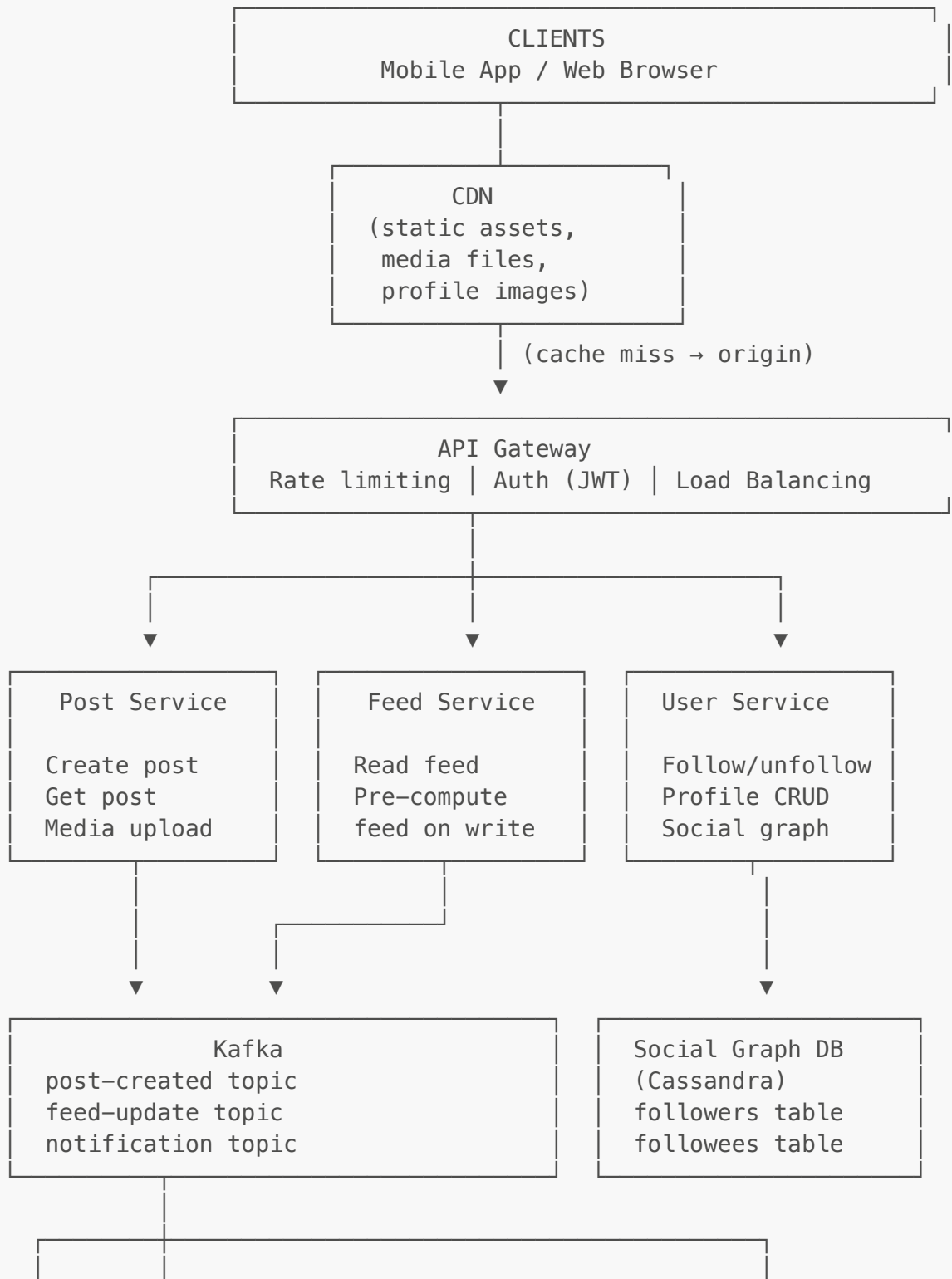
KEY APIS	
POST	/posts Create a post → { postId, mediaUploadUrl }
GET	/feed?userId={id}&cursor={cursor}&limit=20 Paginated home feed (ranked)
POST	/follow/{targetUserId} Follow a user
DELETE	/follow/{targetUserId} Unfollow a user
POST	/posts/{postId}/like Like a post
POST	/posts/{postId}/comments Add a comment
GET	/users/{userId}/posts?cursor=... Profile page – user's own posts

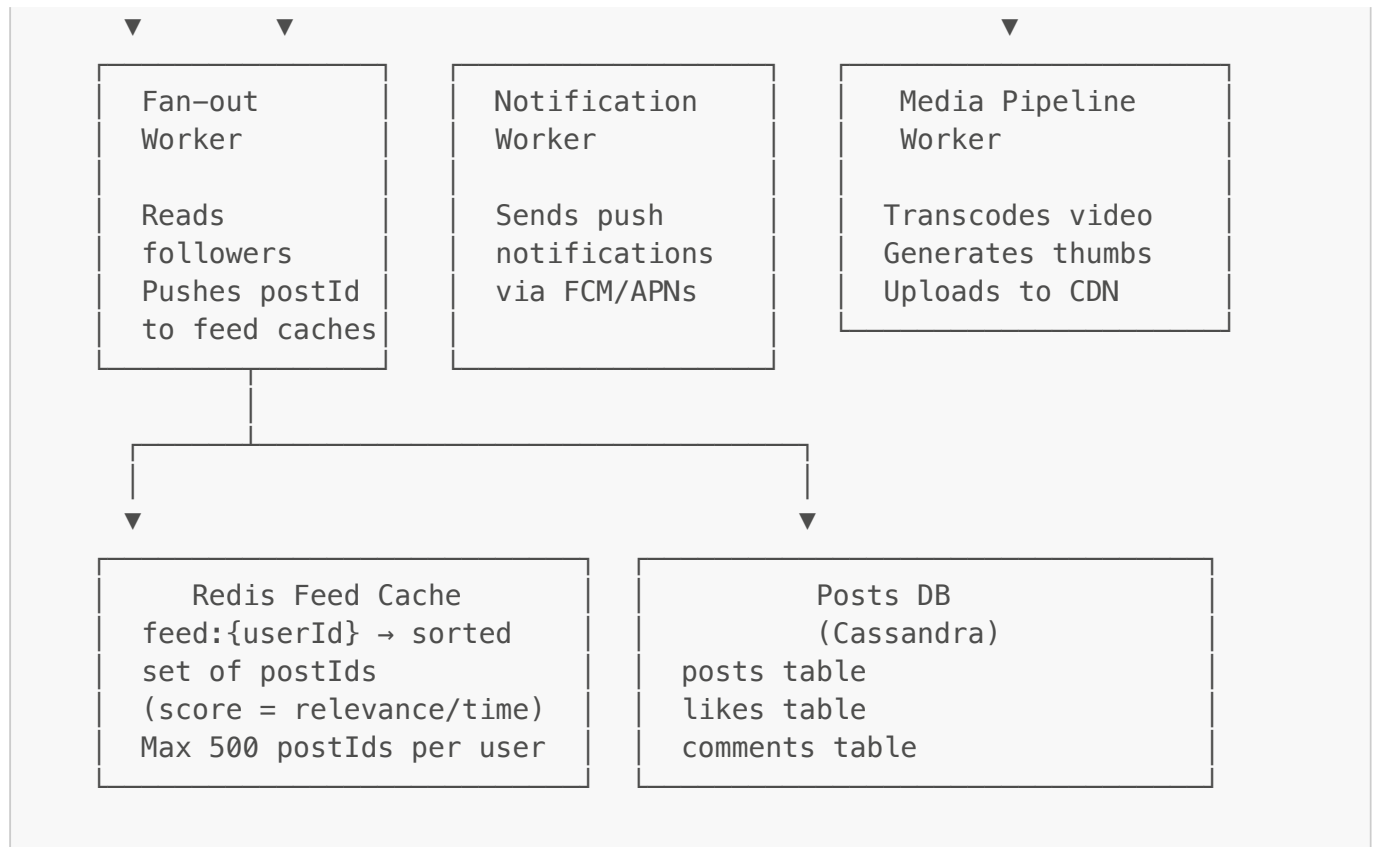
Pagination: cursor-based (not page numbers)

cursor = last seen postId (Snowflake timestamp-encoded)
Prevents "page drift" when new posts arrive
Efficient on Cassandra (range scan by ID)

Phase 5: High-Level Architecture (15 min)

"Let me draw the full architecture. The system has two critical paths: the **write path** (user creates a post) and the **read path** (user loads their feed). Both are distinct in flow and scaling strategy."





Walk through each component:

1. CDN

"All media — images, videos, thumbnails — are served from CDN. This is the single biggest read traffic reducer. Feed reads just get CDN URLs; media itself never hits our servers on reads."

2. Post Service

"Handles post creation. Accepts text + triggers a presigned S3 upload URL for media. Writes to Cassandra. Publishes 'post-created' event to Kafka. That's it. The heavy lifting (fan-out, notifications) happens asynchronously."

3. Feed Service

"Read path for the home feed. First checks Redis feed cache. If populated, returns immediately — sub-millisecond. Cache miss? Falls back to a hybrid pull strategy. More on this in the deep dive."

4. Fan-out Worker

"This is where the magic — and the challenge — lives. Consumes 'post-created' events from Kafka. For each post, fetches the poster's followers and pushes the postId into their feed caches. The celebrity problem lives here."

5. Redis Feed Cache

"Pre-computed sorted sets per user. Key: feed:{userId}, Value: sorted set of (score → postId). Cap at 500 entries per user (evict oldest). The feed read is just a ZRANGE — $O(\log N)$ per user."

Phase 6: Deep Dive — Fan-out Strategy (The Celebrity Problem) (15 min)

"This is the most important and most asked-about section. Let me walk through three approaches and explain why we use a hybrid."

Approach 1: Fan-out on Read (Pull Model)

When user opens their feed:



User follows 500 people
→ 500 DB queries to fetch their latest posts
→ merge $500 \times 20 = 10,000$ posts
→ rank by algorithm
→ return top 20

At 500M DAU requesting feeds:
 $500M \times 500$ queries/req $\times$ peak = BILLIONS of DB reads/sec

Pros:

- ✓ Simple write (just store the post, no fan-out)
- ✓ Always fresh (no stale cache)
- ✓ Works for inactive users (no wasted writes to their cache)

Cons:

- ✗ Extremely slow reads (500 DB queries per user)
- ✗ Cannot meet $< 200ms$ latency requirement
- ✗ DB under crushing read load

VERDICT: Works at small scale. Fails at Instagram scale.

Approach 2: Fan-out on Write (Push Model)

When a user creates a post:

User creates post



Kafka: post-created event



Fan-out Worker fetches ALL follower IDs (from social graph)



For EACH follower, push postId into their Redis feed cache:

`ZADD feed:{followerId} <score> <postId>`



When follower opens their feed:

`ZRANGE feed:{userId} 0 19 (top 20 posts)`

$O(\log N)$ – sub-millisecond!

Feed read: lightning fast (Redis sorted set lookup)

Feed always pre-computed and ready

Pros:

- ✓ Ultra-fast reads ($O(\log N)$ Redis lookup)
- ✓ Meets < 200ms latency easily
- ✓ DB shielded from read traffic

Cons:

× THE CELEBRITY PROBLEM:

Cristiano Ronaldo has 200M followers.

He posts once.

Fan-out worker must write to 200M Redis keys:

$200,000,000 \times \text{ZADD}$ = massive Redis write storm

Time to fan-out: minutes (not seconds)

Followers see the post minutes AFTER he posted it

- × Wasted writes for inactive users
(user hasn't opened app in 3 months – their feed gets all updates, no one reads it)

VERDICT: Fast reads, but celebrity problem is fatal at scale.

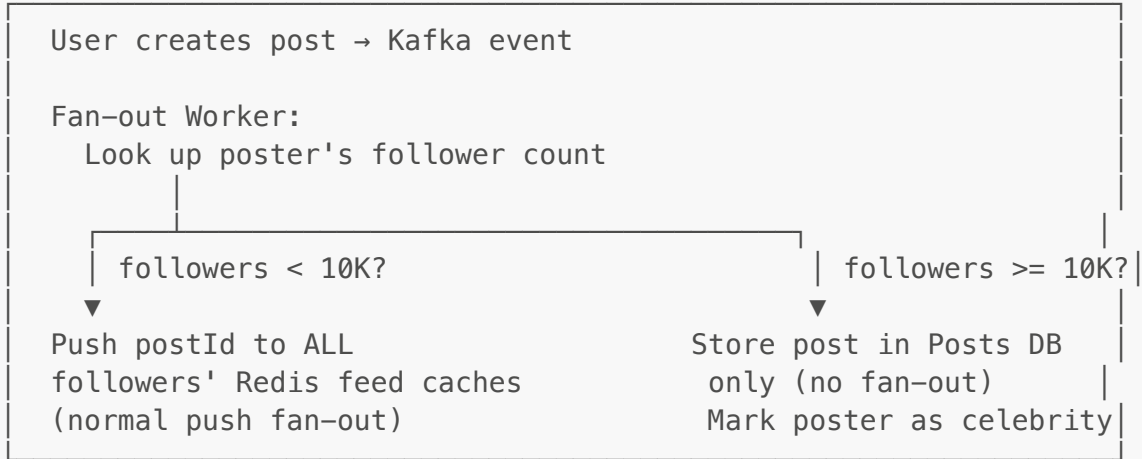
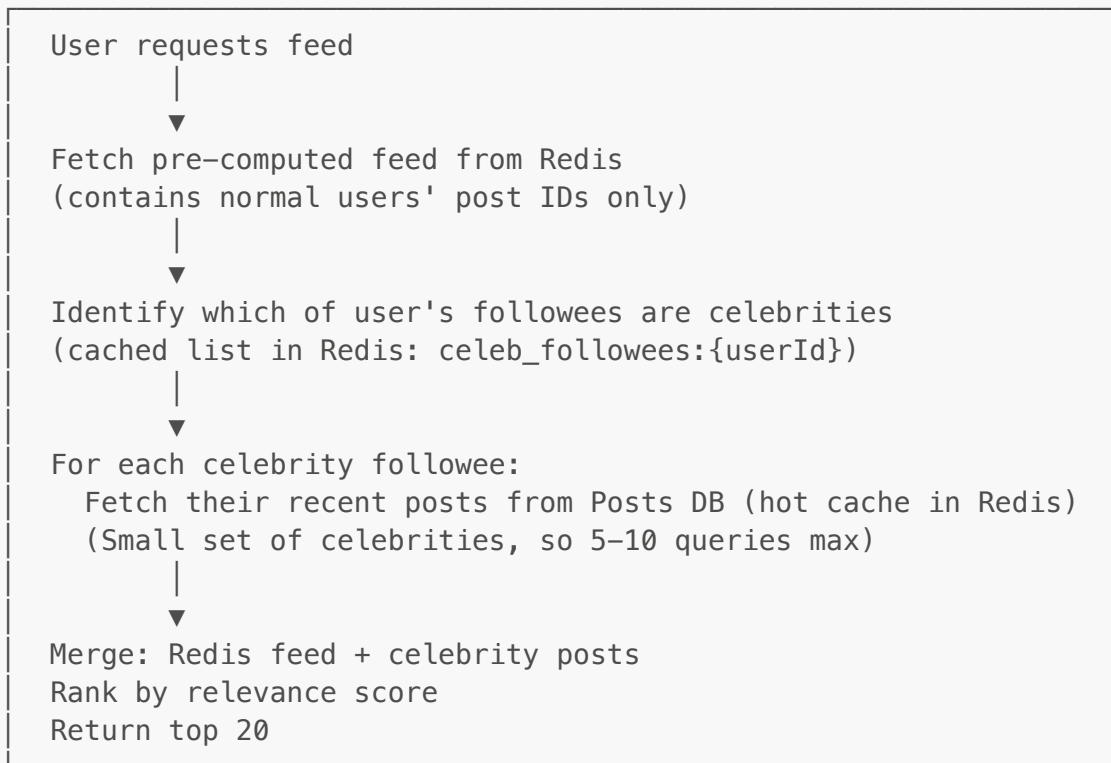
Approach 3: Hybrid Fan-out (What Instagram Actually Uses)

"The solution is elegant: use push for normal users, pull for celebrities."

User types:

Regular user (< 10K followers) → FAN-OUT ON WRITE (push)

Celebrity user (> 10K followers) → FAN-OUT ON READ (pull, merged at feed time)

Post creation flow:**Feed read flow (hybrid merge):****Hybrid Fan-out Visual:**

(Regular user)



Fan-out on Write

Push postId to
all follower feed
caches immediately

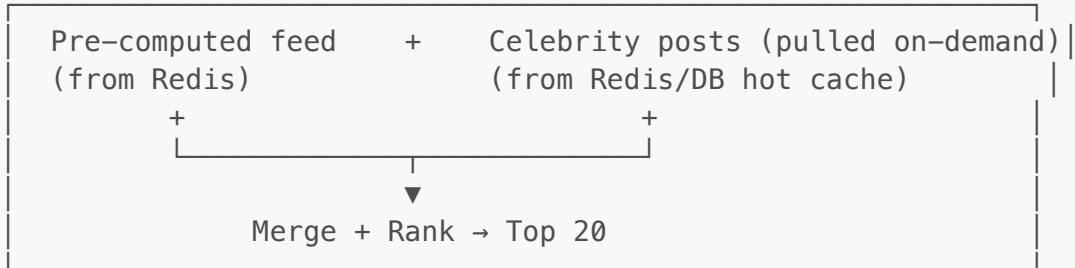
(Celebrity)



Fan-out on Read (hybrid)

Just save post to DB
No fan-out needed
Users pull at read time

At read time:



Why this works:

Regular users: fast push fan-out (thousands of followers = cheap)

Celebrities: avoid the 200M write storm entirely

Celebrity posts are hot (always in Redis) → read is still fast

Phase 7: Feed Ranking (5 min)

"Now let me address ranking. Chronological is simple but a ranked feed is more engaging."

Feed Ranking Signal Categories:

Recency:	How old is the post? (strong signal)
Engagement:	Likes, comments, shares per unit time (strong)
Relevance:	Does user frequently engage with this poster?
	ML model: interaction history → affinity score
Freshness:	Is this the first time user sees this post?

Simple score formula (for interview):

$$\text{score} = \frac{(\text{likes} \times 1) + (\text{comments} \times 3) + (\text{shares} \times 5)}{(\text{hours_since_posted} + 2)^{1.5}}$$

This is similar to Hacker News / Reddit ranking.

Gravity factor (denominator) ensures old posts sink over time.

How it's stored in Redis:

feed:{userId} → sorted set

Key = postId

```
Score = pre-computed relevance score (float)
ZRANGE feed:{userId} 0 19 WITHSCORES → top 20 by score
```

Two-Level Ranking:

Level 1: Candidate generation

Get 200 postIds from Redis feed cache (rough ranking by score)

Level 2: Re-ranking

Fetch metadata for those 200 posts from Redis/Cassandra

Apply more expensive signals (user interaction history, ML)

Re-rank, return top 20

Why two levels?

- Can't run ML on 10,000 candidates in real time
- Redis pre-filters to 200 cheap candidates
- ML re-ranks 200 → 20 (fast: small input)
- Same pattern used by YouTube, TikTok, Twitter

Phase 8: Media Upload Pipeline (5 min)

"Images and videos can't be served from application servers at scale. Here's how the media pipeline works."

User wants to post a photo:

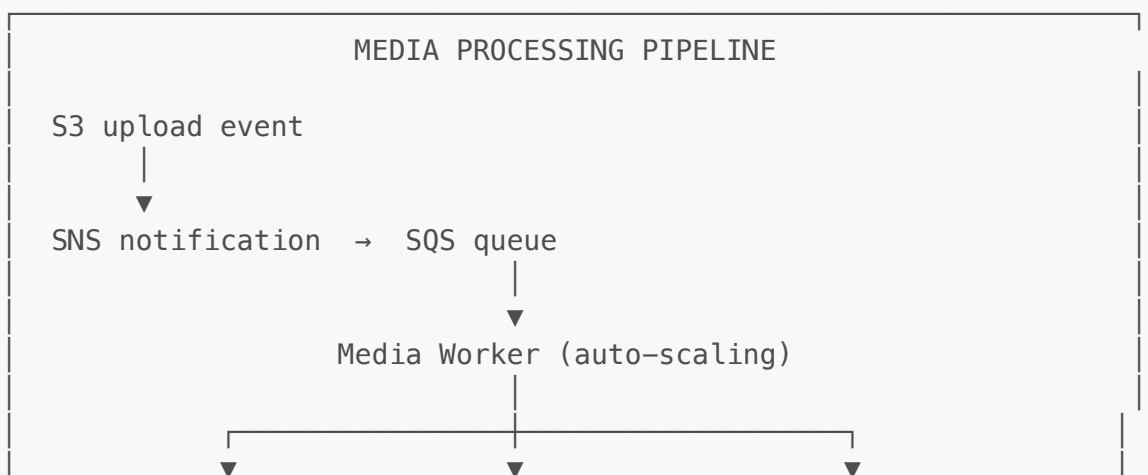
Client requests presigned URL:

POST /posts → server returns { postId, uploadUrl: s3.presigned.url }

Client uploads directly to S3 (bypasses our servers):

PUT {uploadUrl} → raw image goes to S3

S3 triggers processing pipeline:



Resize image (thumbnail, medium, full)	Compress (WebP format)	Blur background (portrait mode)
--	---------------------------	------------------------------------

Upload all variants to S3
 Update post record with CDN URLs
 Publish "post-ready" event to Kafka

CDN pulls from S3 on first access → cached for all future reads.

For videos:

Additional step: Transcoding to multiple resolutions
 (1080p, 720p, 360p) + HLS format for adaptive bitrate streaming.

Phase 9: Scalability & Trade-offs (5 min)

Scaling the Social Graph:

Follow table on Cassandra:

followers_of_X table: (followee_id, follower_id) – who follows X?

followees_of_X table: (follower_id, followee_id) – who does X follow?

Both tables needed – different query patterns.

At 1 trillion follow relationships × 16 bytes = 16 TB

Partitioned by followee_id / follower_id respectively.

Reads: O(1) partition lookup, fast range scan.

Scaling the Fan-out Worker:

For normal users (fan-out on write):

If user has 10,000 followers:

10,000 ZADD commands to Redis

Batch them: Redis pipeline (1 round trip, 10K commands)

Time: ~50ms per fan-out batch

Multiple fan-out workers in parallel (Kafka partitioning)

Each worker handles a subset of followers

(partition followers by userId hash)

Redis Feed Cache Sizing:

500M users × 500 postIds × 8 bytes/postId = 2 TB in Redis
 Redis cluster: 10 nodes × 200 GB = 2 TB with headroom

TTL: evict feeds for users inactive > 7 days
 (no point pre-computing for dormant accounts)

Cap: max 500 postIds per feed (ZADD with eviction of oldest)

Handling Unfollow:

User unfollows celebrity:
 Remove from celeb_followees:{userId}
 Next feed read naturally excludes them (no pull for unfollowed celeb)

User unfollows regular person:
 Mark posts in feed cache as "stale for this user" OR
 Lazy cleanup: just don't include posts from unfollowed users at merge
 time
 (Check "is user still following this poster?" before including in feed)

LLD Interview (45-60 min)

Phase 1: Scope (5 min)

Q: Which service to design in detail?
 A: Fan-out service + Feed service (the most interesting parts)

Q: Depth expected on Redis data structures?
 A: Yes – explain sorted sets, hash maps, TTL patterns

Q: Ranking algorithm internals?
 A: Walk through the two-level approach

Phase 2: Domain Models (10 min)

Post Table (Cassandra):

Post (Cassandra)	
post_id	Snowflake ID (partition key)
user_id	Creator
caption	Text content
media_urls	List<CDN URL> – images/video

media_type	IMAGE / VIDEO
like_count	Counter (eventually consistent)
comment_count	Counter
created_at	Timestamp (from Snowflake ID – free!)
location	Optional geotag
hashtags	Set<String>

Follow Table (Cassandra — two tables):

Table: followers_by_user (who follows user X?)
 Partition key: followee_id
 Clustering key: follower_id
 → Query: "give me all followers of user X" → fast
 → Used by: fan-out worker

Table: followees_by_user (who does user X follow?)
 Partition key: follower_id
 Clustering key: followee_id
 → Query: "give me everyone user X follows" → fast
 → Used by: pull-model feed, follow suggestions

Redis Key Schema:

REDIS KEY SCHEMA	
feed:{userId}	Sorted set: score → postId Max 500 entries, TTL 7 days
post:{postId}	Hash: all post metadata TTL: 24h (hot posts stay cached)
celeb_followees:{userId}	Set of celebrity userIds this user follows (pulled at feed time)
user_profile:{userId}	Hash: username, follower_count, is_celebrity, avatar_url TTL: 1h (semi-static)
likes:{postId}	Counter (INCR/DECR) Flushed to Cassandra periodically
user_liked:{userId}:{postId}	Boolean – has user liked this post? TTL: 7 days

Phase 3: Fan-out Worker — Full Flow (10 min)

FanoutWorker receives Kafka event:

```
{
  eventType: "POST_CREATED",
  postId: "1234567890",
  userId: "alice",
  followerCount: 847,
  score: 100.0,
  createdAt: 1722000000
}
```

Is poster a celebrity? (follower_count > CELEBRITY_THRESHOLD)

Celebrity

Normal user

Fetch ALL follower IDs from Cassandra:
SELECT follower_id FROM followers_by_user
WHERE followee_id = alice

Batch followers into chunks of 1,000
Spawn parallel goroutines for each chunk:
for each followerId in chunk:
 ZADD feed:{followerId} score postId
 ZREMRANGEBYRANK feed:{followerId} 0 -501 ← trim to 500

Mark postId as "ready for feed"

▼ (celebrity path)

Only store post in Posts DB + Redis post cache.
Update user_profile:{celebrity} with latest postId.
(Followers fetch on-demand at feed read time)

Fan-out Worker Concurrency:

Single celebrity post → 10M followers to process

Without parallelism: 10M writes sequentially → minutes

With parallelism:

Kafka partition by followerId % N_partitions

100 fan-out workers consume in parallel

Each worker: 10M / 100 = 100K followers

100K × 0.1ms/ZADD = 10 seconds (if N=100 workers)

For 200M followers (Cristiano Ronaldo):
 $200M / 100 \text{ workers} = 2M \text{ per worker}$
 $2M \times 0.1ms = 200 \text{ seconds} \rightarrow \text{STILL too slow!}$

Solution: Celebrity threshold $\rightarrow$ pull model for them.
 Don't even attempt fan-out for mega-celebrities.

Phase 4: Feed Service — Full Read Flow (10 min)

```

GET /feed?userId=bob&cursor=null&limit=20
  |
  ▼
FeedService.getFeed(userId=bob, cursor=null, limit=20)
  |
  ▼
Step 1: Fetch pre-computed feed from Redis
  ZRANGE feed:bob -inf +inf BYSCORE WITHSCORES LIMIT 0 200 REV
  (get top 200 posts by score, descending)
  → Returns: [(score1, postId1), (score2, postId2), ...]
  |
  ▼
Step 2: Get celebrity followees of bob
  SMEMBERS celeb_followees:bob
  → Returns: {cristiano, taylorswift, nasa}
  |
  ▼
Step 3: For each celebrity, fetch their recent posts
  GET user_profile:cristiano → { latestPostIds: [p1, p2, p3...] }
  (Redis hash, pre-populated when celebrity posts)
  → Returns small set of recent post IDs per celebrity
  |
  ▼
Step 4: Merge + deduplicate
  candidatePosts = precomputed_feed + celebrity_posts
  Remove any postIds already seen (cursor-based pagination)
  |
  ▼
Step 5: Fetch post metadata for candidates
  Multi-GET from Redis post cache:
  MGET post:p1 post:p2 post:p3 ...
  Cache miss? → fetch from Cassandra, backfill Redis cache
  |
  ▼
Step 6: Re-rank (ML-lite)
  For each post, compute affinity score:
    affinity = base_score × interaction_history_factor
  Sort by final score
  Take top 20
  
```


↓
▼
Step 7: Return paginated response

```
{
  posts: [post1, post2, ... post20],
  nextCursor: lastPostId ← used for "load more"
}
```

Phase 5: Notification Service (5 min)

Events that trigger notifications:

- New follower
- Post liked (by someone you know)
- Comment on your post
- Mention in a comment
- New post from followed user (optional – configurable)

Flow:

```
Kafka: notification-events topic
{ type: LIKE, actorId: carol, targetUserId: bob,
  postId: p123, timestamp: ... }
```

Notification Worker:

1. Check user's notification preferences (Redis/DB)
2. Check if user is currently active (presence in Redis)
 - If active: push via WebSocket/SSE (in-app)
 - If offline: push via FCM (Android) or APNs (iOS)
3. Store notification in Notifications DB (Cassandra)
4. Increment unread badge count: INCR notif_count:{userId}

Notification fanout for "new post" type:

Same celebrity problem applies!

Taylor Swift posts → 100M notifications

Solution:

```
Normal user: send push to all followers immediately
Celebrity:   sample notification – send to most-engaged
            followers only (not all 200M!)
            Instagram only notifies a subset for large
            accounts to avoid notification fatigue
```

Phase 6: Like Counter — High-Frequency Writes (5 min)

"Like counts are interesting — a viral post might get 100K likes per second. Writing each like directly to Cassandra would saturate it."

Problem: 100K likes/sec on one post

Naive: UPDATE posts SET like_count = like_count + 1 WHERE post_id = X

At 100K/sec: Cassandra write storm on one partition (hot key)

Solution: Counter service with Redis buffering

User likes post → INCR likes:{postId} in Redis
Redis handles 100K+ incr/sec easily (in-memory, single-key)

Background job every 5 seconds:

Read Redis counter: GET likes:{postId}

Write to Cassandra: SET like_count = {value}
(not increment, but set – idempotent)

User reads like count:

Check Redis first (authoritative during session)
Fall back to Cassandra

Did user already like this post? (idempotency check)

SETNX user_liked:{userId}:{postId} "1"

If returns 0 → user already liked → reject duplicate

If returns 1 → new like → proceed

Phase 7: Design Patterns (5 min)

1. CQRS — Command Query Responsibility Segregation

Write path (Command):

User creates post → PostService → Cassandra (write-optimized)

Kafka event → Fan-out worker → Redis feed caches

Read path (Query):

FeedService → Redis (read-optimized sorted sets)

Post metadata → Redis hot cache → Cassandra fallback

CQRS insight:

The write and read representations are DIFFERENT.

Posts stored in Cassandra (normalized, write-optimized)

Feed served from Redis (denormalized, read-optimized)

Write path pushes changes to read model asynchronously

2. Observer Pattern — Kafka as event bus

PostCreatedEvent (subject) → multiple observers:

FanoutWorker	subscribes → updates Redis feed caches
NotificationWorker	subscribes → sends push notifications
MediaWorker	subscribes → transcodes video if needed
SearchIndexer	subscribes → indexes post for search
AnalyticsWorker	subscribes → tracks engagement metrics

Decoupled: adding a new observer never changes PostService.
Each worker scales independently.

3. Strategy Pattern — Feed ranking

FeedRankingStrategy (interface):

rank(posts: List<Post>, userId: String) → List<Post>

ChronologicalRankingStrategy	implements FeedRankingStrategy
EngagementRankingStrategy	implements FeedRankingStrategy
MLRankingStrategy	implements FeedRankingStrategy

FeedService uses strategy (injected at construction).

A/B testing: route 10% to ML strategy, 90% to engagement.

4. Write-Behind Cache Pattern (Redis for likes)

User action (like) → Redis counter (immediate, fast)

Background writer → persists to Cassandra asynchronously

User sees up-to-date count (from Redis).

DB sees batch updates (efficient).

Trade-off: if Redis crashes before flush → temporary count loss.

Solution: Redis AOF persistence + periodic Cassandra flush.

Common Questions & Answers

Q1: How do you handle a user with both a large following AND many celebrity followees?

Example: A large influencer (500K followers) who follows 20 celebrities.

Write path (they post):

500K followers → fan-out on write (they're below celebrity threshold)
Each follower gets the postId pushed to their feed cache

Read path (they load their feed):

Their pre-computed feed (from people they follow who are non-celebrities)
+ pull from the 20 celebrities' recent posts (hybrid merge)

No problem – 20 celebrity lookups is cheap

Both paths handled correctly by the hybrid model.

Q2: What happens when the user's feed cache is cold (evicted)?

Cache miss scenario:

User hasn't opened app in 8 days

Their feed cache TTL expired

→ ZRANGE feed:{userId} returns empty

Cold start:

Option A: Rebuild from scratch

Query Posts DB for all followees' recent posts

Rank them all, take top 200, populate Redis

Expensive but only happens once per cold user

Option B: Lazy population

Return empty feed immediately with a "building feed" indicator in the response

Async job rebuilds cache (takes 1–2 seconds)

Next request (2s later) gets full feed

Tradeoff: First screen is empty briefly

Instagram uses: show skeleton loader → async build → populate
Most users don't notice the 1–2 second delay.

Q3: How do you handle duplicate posts in the feed?

Can happen when:

- Fan-out worker retries (at-least-once delivery)
- User following someone just unfollowed then re-followed

Deduplication strategy:

Redis sorted set is idempotent:

ZADD feed:{userId} score postId

→ If postId already in set, updates score (no duplicate)

At read time:

Set of seen postIds in this response generation session

Skip any postId already included

Q4: How do you implement "mute user" without removing follow?

Mute: user follows X but doesn't want to see their posts in feed
(Follow count still increases – social courtesy)

Implementation:

```
Store muted users: SET muted_by:{userId} → {mutedUserId}
Key: muted:{userId} = set of muted userIds
```

```
Fan-out worker: before writing to feed cache
  if mutedUserId in SMEMBERS muted:{followerId}: skip
  (muted users' posts not pushed to feed cache)
```

```
OR lazy check at read time:
  After fetching feed candidates, filter out muted users
  (simpler, but means wasted postIds in cache)
```

Q5: How does close-friends / restricted sharing work?

Post visibility tiers:

PUBLIC: anyone can see

FOLLOWERS_ONLY: only followers can see

CLOSE_FRIENDS: only selected users (Instagram Close Friends feature)

PRIVATE: only the user

Close Friends implementation:

```
Store: close_friends:{userId} = Set of close friend userIds
```

```
When close-friends post created:
```

```
  Fan-out ONLY to userIds in close_friends set
  (not all followers)
```

```
  Mark postId with visibility flag = CLOSE_FRIENDS
```

```
Read time enforcement:
```

```
  If post.visibility == CLOSE_FRIENDS:
```

```
    Check: is reader in poster's close_friends set?
```

```
    NO → redact from response
```

```
    YES → include
```

```
Double enforcement: fan-out filters + read-time check
```

Q6: User deletes a post — how do you remove it from all feed caches?

This is the classic cache invalidation trap.

Naive approach: scan all 500M feed caches and remove the postId

WRONG: 500M ZREM commands for one deletion
Completely infeasible – would take hours

Correct approach: Tombstone + lazy filter at read time

Step 1: Mark post as deleted in Posts DB
UPDATE posts SET status = DELETED WHERE post_id = X
SET deleted_posts:{postId} "1" EX 604800 (Redis, 7d TTL)

Step 2: Publish "post-deleted" event to Kafka
Consumers: notification service, search indexer

Step 3: At feed read time (FeedService):
For each postId from Redis sorted set:
Check: EXISTS deleted_posts:{postId}
If exists → skip (filter out)
If not → include in response

Why TTL of 7 days on deleted_posts key?
Feed cache max holds 500 posts, TTL 7 days.
After 7 days, deleted post would have aged out anyway.
Tombstone can safely expire after 7 days.

Result:

No mass cache invalidation
Deleted posts disappear from feeds within seconds (lazy filter)
Read overhead: O(1) Redis lookup per postId (EXISTS = fast)

Q7: User A blocks User B — how do you handle it?

Block has two effects:

1. A should not see B's posts in their feed
2. B should not see A's posts in their feed (bidirectional)

Implementation:

Store: blocked:{userId} = Set of blocked userIds
Also: blocked_by:{userId} = Set of users who blocked them
(Two sets – query both directions)

At fan-out write time:
Before pushing postId to follower's feed cache:
SISMEMBER blocked:{followerId} {posterId}

If true → skip (don't push blocked user's posts)

At feed read time:

For each postId in feed candidates:

Fetch post.userId

SISMEMBER blocked:{requestingUser} {post.userId}

If true → filter out (defense in depth)

Why both write-time AND read-time checks?

Write-time: performance (skip pushing to blocked users)

Read-time: safety net (catches race conditions,
handles posts already in cache before block)

When block happens mid-session:

Posts already in A's Redis feed cache from B still exist.

Read-time filter catches them immediately.

No need to scan and purge the feed cache.

Q8: Redis goes down — what happens to all pre-computed feeds?

This is the "single point of failure" trap.

Impact:

All pre-computed feed caches lost.

500M users request their feed → all are cache misses.

Cassandra gets hammered with 500M simultaneous fan-out reads.

→ Thundering herd → DB overload → cascades into outage.

Mitigation layers:

Layer 1: Redis Cluster (not single Redis)

Shard across 10 nodes — only 1/10 of feeds lost per node

Replica per shard (Redis Sentinel / Cluster replication)

Node failure: replica promotes in seconds

Layer 2: Graceful degradation on full cluster failure

Fall back to fan-out on READ mode temporarily

Route feed reads to Posts DB (Cassandra)

Serve degraded but correct feed

Latency degrades: 200ms → 1-2s (acceptable for DR)

Layer 3: Circuit breaker on Feed Service

Redis unavailable → open circuit → skip cache check

Go directly to DB, return whatever Cassandra can serve

Layer 4: Progressive rebuild (not all at once)

When Redis recovers, DON'T immediately rebuild all feeds

Rebuild on-demand as users request their feeds

Prevents DB stampede during recovery

Q9: Thundering herd — everyone opens the app after a celebrity posts

Scenario:

Taylor Swift posts at midnight on New Year.

100M followers all open the app within 60 seconds.

Each requests GET /feed → all need Taylor's latest post.

Taylor is a celebrity → her post is NOT fan-out to individual caches.

Feed service pulls from celebrity post cache: `post:{taylorId}:recent`

100M simultaneous requests all hit:

GET `post:{taylorId}:recent` in Redis → HOT KEY problem

A single Redis key can handle ~100K reads/sec

100M req in 60s = ~1.7M req/sec → Redis node overwhelmed

Solutions:

Solution 1: Local in-process cache

Each Feed Service instance caches celeb posts for 1s

1000 Feed Service instances × 100K req/sec each

Only 1000 Redis requests/sec total (one per instance/sec)

→ 1000x reduction in Redis load

Solution 2: Read replicas for hot celebrity keys

Detect hot keys (requests > threshold in 1s)

Replicate key across multiple Redis nodes

Route reads round-robin across replicas

Solution 3: Proactive fan-out for "event posts"

Detect high-engagement posts within first 60 seconds

Retroactively fan-out to top active followers' caches

(Instagram actually does this for viral posts)

Q10: Fan-out worker falls behind — Kafka consumer lag

Scenario:

Burst of 1M posts in 10 minutes (viral event, breaking news)

Fan-out worker can process 100K posts/min

Kafka backlog grows: workers are 9 minutes behind

Followers see stale feeds while backlog drains

Detection:

Monitor Kafka consumer group lag: `kafka consumer-groups --describe`

Alert when lag > threshold (e.g., > 100K messages)

Mitigation:

Response 1: Auto-scale fan-out workers

Kafka consumer group: add more consumers = more parallel
Consumers $\leq$ partitions (so pre-partition generously: 100)
Burst to 100 workers $\rightarrow$ 100 $\times$ throughput

Response 2: Priority queuing

Two Kafka topics: high-priority, low-priority
Recent posts (< 5 min old) $\rightarrow$ high-priority topic
Older posts $\rightarrow$ low-priority topic
Workers drain high-priority first

Response 3: Skip stale fan-outs

If post is > 30 min old when worker processes it:
Skip the fan-out entirely
Users will see it via pull-fallback or next cold rebuild
(Nobody cares about a post that's 2 hours late in a feed)

Response 4: Feed service fallback

If fan-out lag detected: Feed Service adds pull-layer
Fetch recent posts directly from Posts DB for all
followers (hybrid mode activated temporarily)

Key insight:

Eventual consistency is acceptable for feeds (SLA: < 5 sec normally)
During lag events: feeds degrade gracefully, not catastrophically.

Q11: Why Cassandra over MySQL for posts and follow graph?

This is asked to test your DB selection reasoning.

Why NOT MySQL (or PostgreSQL)?

Posts table: 100M writes/day = ~1,200 writes/sec
MySQL handles ~10K writes/sec on one node $\rightarrow$ fine actually
But: 10 billion posts total = ~5 TB of post data
MySQL sharding at this scale = huge operational complexity

Follow graph: 1 trillion rows = 16 TB in a follows table
MySQL cannot hold this on one node $\rightarrow$ sharding required
SQL joins across shards = impossible at query time

Why Cassandra for posts:

- Partition key = post_id (Snowflake) $\rightarrow$ even distribution
- Linear horizontal scaling: add nodes $\rightarrow$ add capacity

- Tunable consistency: write with quorum, read fast
- Time-series fit: post_id is time-ordered → range scans
- No single points of failure

Why Cassandra for follow graph:

- 1 trillion rows: Cassandra handles petabyte scale
- Partition by followee_id → O(1) follower lookup
- Denormalized (two tables) = no joins needed
- High write throughput for follow/unfollow

What MySQL IS good for in this system:

- User profiles (low write volume, needs ACID for uniqueness)
- Billing / subscription data (ACID transactions required)

Q12: Why Redis Sorted Sets for feeds — not a List or Hash?

This tests whether you actually know Redis data structures.

Why NOT a List (LPUSH/LRANGE)?

- No deduplication: same postId can be added twice
- Insertion order only: can't re-rank by score after the fact
- No efficient "insert by score": must scan to find position

Why NOT a Hash (HSET)?

- No ordering: HGETALL returns unordered
- Must fetch all entries and sort in application layer
- O(N) for any "top K" query

Why Sorted Set (ZADD/ZRANGE)?

- Deduplication: postId is the member (unique within set)
- Score = relevance/timestamp: ZRANGE by score = ranked feed
- O(log N) insert, O(log N + K) for top-K query
- ZADD NX: add only if not exists (skip duplicates)
- ZREMRANGEBYRANK: trim to 500 entries in one command
- ZRANGEBYSCORE: efficient cursor-based pagination

Sorted Set is precisely the right data structure for feeds:
ordered, unique, scored, trimable, range-queryable

Cheat Sheet

Key Numbers to Remember

DAU: 500M
Posts per day: 100M (~1,200/sec)
Feed reads: 10 billion/day (~115,000/sec)
Read:Write ratio: 100:1
Celebrity threshold: 10,000 followers
Feed cache size: Max 500 postIds per user
Feed cache TTL: 7 days inactivity → evict
Like counter flush: Every 5 seconds Redis → Cassandra
Feed latency target: < 200ms

Fan-out Strategy Decision

Poster has < 10K followers → Fan-out on WRITE
Push postId to all followers' feed caches

Poster has >= 10K followers → Fan-out on READ (celebrity path)
Only store post in DB
Followers pull at read time

At feed read time: Pre-computed feed (normal users)
+ On-demand pull (celebrities)
+ Merge + Re-rank

Database Choices

Posts: Cassandra (write-heavy, time-series by postId)
Follow graph: Cassandra (two tables: followers_by_X, followees_by_X)
User profiles: PostgreSQL or Cassandra (low write volume)
Feed cache: Redis sorted sets (score = relevance)
Like counters: Redis (buffered, flushed to Cassandra)
Media files: S3 + CDN
Sessions/auth: Redis (JWT allowlist / refresh tokens)

Ranking Signals

Recency: Age of post (gravity decay over time)
Engagement: likes×1 + comments×3 + shares×5 (per hour since posted)
Affinity: How often viewer interacts with this poster
Freshness: Not yet seen by this user
Two levels: 1) Redis pre-filter → 200 candidates
2) ML re-rank → 20 final posts

Design Patterns Used

CQRS: Separate write (Cassandra) and read (Redis) models
 Observer: Kafka event bus – PostCreated → multiple consumers
 Strategy: Pluggable ranking (chronological vs ML)
 Write-Behind: Redis like counter → async flush to Cassandra
 Factory: FanoutStrategyFactory.get(followerCount) → strategy
 Event Sourcing: Post history derivable from Kafka event log

The Celebrity Problem — Summary

Problem: Celebrity with 200M followers posts
 → 200M Redis ZADD commands
 → Takes minutes, not seconds
 → Followers see post late

Solution: Hybrid fan-out
 Regular user posts: Write to all follower caches immediately
 Celebrity posts: Store in DB only, followers pull at read time
 At read time: Merge pre-computed + celebrity-pulled + re-rank

Why it works:
 Celebrity posts are HOT (always in Redis hot cache)
 Pulling from 5–10 celebrities ≈ 5–10 Redis lookups (fast!)
 No 200M write storm

Must-Know Concepts

Fan-out on Write: Push to caches at post time (fast reads, slow writes)
 Fan-out on Read: Pull from DB at read time (fast writes, slow reads)
 Hybrid fan-out: Best of both – write for normal, read for celebrity
 Snowflake IDs: Time-sortable IDs (no ORDER BY timestamp needed)
 Cursor pagination: Better than page numbers for live feeds
 Write-behind cache: Absorb write spikes in Redis, flush to DB async
 CQRS: Write model ≠ read model (different storage)
 Two-level ranking: Cheap pre-filter → expensive ML rerank

Final Tips

For HLD Interview

1. **Lead with the scale constraint:** "100:1 read-write ratio and 200ms latency — this forces pre-computed Redis feeds over on-demand DB queries."
2. **Name the celebrity problem before interviewer asks** — shows depth
3. **Walk through all 3 fan-out approaches** (read, write, hybrid) — structured thinking
4. **Mention Kafka** between post creation and fan-out — async decoupling

5. Explain the two-level ranking — candidate generation → re-ranking

For LLD Interview

1. **Two Cassandra tables for follows** — both lookup directions matter
2. **Redis key schema** — know what goes in sorted sets vs hashes vs sets vs strings
3. **Write-behind cache for counters** — absorb like spikes in Redis
4. **CQRS** — the write model and read model are deliberately different
5. **Fan-out threshold** — be specific about the number (10K followers)

Red Flags to Avoid

- ✗ "We'll query all followees' posts on each feed load" — 500 DB queries per user
- ✗ Fan-out on write for ALL users including celebrities — 200M write storm
- ✗ Using SQL for follow graph at 1 trillion relationships
- ✗ Serving media directly from application servers (not CDN)
- ✗ Storing like counts in DB with row-level locks — hot key problem
- ✗ Page-number pagination for feeds — "page drift" as new posts arrive

Green Flags to Aim For

- ✓ "The design-defining constraint is the 100:1 read-write ratio..."
- ✓ "We need to handle the celebrity problem — someone like Cristiano Ronaldo..."
- ✓ "For celebrities we flip to fan-out on read, pull at feed time..."
- ✓ "Redis sorted sets let us store pre-ranked feeds with $O(\log N)$ reads"
- ✓ "Like counters are write-heavy — we buffer in Redis and flush async"
- ✓ "Two-level ranking: Redis pre-filter to 200 candidates, ML re-ranks to 20"

Comparison: Social Media vs WhatsApp vs Google Docs

Dimension	Instagram/Facebook	WhatsApp	Google Docs
Core challenge	Feed generation + fan-out	Real-time message delivery	Conflict resolution (OT)
Connection	REST + WebSocket (notifications)	WebSocket	WebSocket
Famous problem	Celebrity fan-out	Cross-server routing	OT transform algorithm
Primary cache	Redis feed sorted sets	Redis presence + routing	Redis canonical copy

Dimension	Instagram/Facebook	WhatsApp	Google Docs
Write pattern	Async fan-out via Kafka	Sync delivery via Kafka	Sync op apply via OT engine
Read pattern	Pre-computed feed from Redis	Pull from Cassandra	Live from Redis canonical
Media	S3 + CDN (async transcoding)	S3 + CDN (encrypted)	S3 (version snapshots)
Consistency	Eventual (feed can lag ~5s)	At-least-once + dedup	Strong within session
Scale trick	Hybrid fan-out (write/read)	Kafka partitioned by recipientId	Sticky routing by docId

CONVERSATIONAL INTERVIEW SCRIPT

"Why Cassandra for the follow graph and posts — not MySQL?"

Say this out loud exactly like this. Own the reasoning before the interviewer asks.

"I want to explain my DB choice here rather than just dropping 'Cassandra' — because the reasoning is the real answer.

The follow graph has 1 trillion rows. 1 trillion × 16 bytes = about 16 TB. A single MySQL server comfortably holds maybe 2–4 TB before you hit performance walls. So MySQL on one machine is physically out.

Next question: can I shard MySQL? Yes — but sharding MySQL is brutal. You're manually deciding which server holds which rows, writing routing logic, and the moment you query across shards — 'give me all followers of Bob' — you fan out to multiple servers and merge in application code. At petabyte scale, that's unmaintainable.

Cassandra solves this with partition keys. I set followee_id as the partition key, and Cassandra automatically puts all of Bob's followers on the same node. My fan-out worker asks for Bob's followers — one node, one fast range scan. Done."

CASSANDRA PARTITION LAYOUT
Table: followers_by_user

Partition key = followee_id

```
Node-1: followee=alice      → [f_1, f_2, f_3 ... f_847]
Node-2: followee=bob       → [f_1, f_2, f_3 ... f_320K]
Node-3: followee=cristiano → [f_1, f_2, f_3 ... f_200M]
                                (spread across virtual nodes)
```

Fan-out worker query:

```
SELECT follower_id FROM followers_by_user WHERE followee_id = 'alice'
```

→ hits ONE partition on ONE node → $O(1)$ locate + sequential scan
 → returns 847 rows in ~2ms regardless of total table size (1 trillion)

"The other reason: write throughput. We write 100M posts per day and every follow/unfollow is a write. Cassandra's LSM-tree structure absorbs high write rates much better than MySQL's B-tree.

For user profiles — uniqueness of usernames, account creation — I'd use PostgreSQL there. You need ACID for that. But for the massive-scale data with known query patterns and no cross-entity joins, Cassandra is the right call."

"What does fan-out mean? And why do we need a Fan-out Worker?"

"Fan-out just means: one thing spreads out to many places. Cristiano Ronaldo creates one post. 200 million people need to see it in their personal feed. That expansion — one post becoming 200M feed entries — is the fan-out.

Now here's the problem: if you try to do that synchronously — meaning, on the same thread that's handling Cristiano's POST request — you'd have to find all 200M followers and write to their feeds before returning a response. That takes minutes. His app would spin forever.

The Fan-out Worker decouples creation from propagation."

WRITE PATH — decoupled via Kafka

Cristiano's phone

| POST /posts { caption, mediaUrl }



Post Service

1. INSERT INTO posts (Cassandra) ← ~5ms
 { postId, userId, caption,
 mediaUrl, createdAt }

2. PUBLISH to Kafka ← ~3ms
 topic: post-created
 payload: {
 postId: "p_abc123",
 userId: "cristiano",
 followerCount: 200000000,
 initialScore: 100.0,
 createdAt: 1722000000
 }

3. Return 201 Created ← ~2ms

Cristiano sees 
 "Posted!" in 10ms



← post creation is DONE

| Kafka event (background, async)
▼

Fan-out Worker (100 instances running in parallel)

STEP 1: Stream follower IDs from Cassandra

```
SELECT follower_id FROM followers_by_user
```

```
WHERE followee_id = 'cristiano'
```

```
LIMIT 1000
```

```
(paginated – streams 200M rows in batches)
```

STEP 2: For each batch of 1000 followers:

```
Redis pipeline {           ← 1 network round-trip
```

```
  ZADD feed:alice    100.0  "p_abc123"
```

```
  ZADD feed:bob      100.0  "p_abc123"
```

```
  ZADD feed:carol    100.0  "p_abc123"
```

```
  ...× 1000
```

```
  ZREMRANGEBYRANK feed:alice 0 -501  ← keep max 500
```

```
  ZREMRANGEBYRANK feed:bob   0 -501
```

```
  ...× 1000
```

```
}
```

100 workers × 2M followers each = done in ~5–10 seconds

(acceptable – eventual consistency SLA is < 5 sec for

normal users; celebrities use pull model instead)

"The key insight is eventual consistency is fine here. If a post appears in your feed 3 seconds after it was created, you won't notice."

"Explain the Redis sorted set — what ZADD and ZRANGE actually do."

"A Redis sorted set is a leaderboard. Every entry has a member — in our case a postId — and a score, a float that represents relevance. The set stays automatically sorted by score. No duplicates. If you ZADD the same postId twice, it just updates the score.

ZADD = add this member with this score. ZRANGE = give me members from position X to Y."

REDIS SORTED SET: feed:alice

Fan-out worker writes (one pipeline call per batch):

```
ZADD feed:alice 99.1 "post:bob:1234"
```

```
ZADD feed:alice 87.3 "post:carol:5678"
```

```
ZADD feed:alice 76.8 "post:dave:9012"
```

```
ZADD feed:alice 55.2 "post:eve:3456"
```

```
ZADD feed:alice 34.9 "post:frank:7890"
```

Internal state – always maintained in sorted order:

RANK	SCORE	MEMBER
0	99.1	post:bob:1234 ← shown first
1	87.3	post:carol:5678
2	76.8	post:dave:9012
3	55.2	post:eve:3456
4	34.9	post:frank:7890 ← shown last

Alice opens app:

```
ZRANGE feed:alice 0 19 REV WITHSCORES
```

→ returns top 20 postIds in order, $O(\log N)$, ~0.5ms

Score update (post gets more likes, re-scored to 94.0):

```
ZADD feed:alice 94.0 "post:carol:5678"
```

→ sorted set re-sorts automatically – carol moves to rank 1

Trim to 500 after new entry:

```
ZREMRANGEBYRANK feed:alice 0 -501
```

→ removes entry at rank 500 and below (evict oldest/lowest)

Why this beats a List or Hash:

List (LPUSH): no deduplication, insertion-order only, can't re-rank

Hash (HSET): unordered, must fetch ALL and sort in app code – $O(N)$

Sorted Set: deduplicated, always sorted, top-K in $O(\log N + K)$

"This is why we call it pre-computed. The fan-out worker does all the ZADD writes when the post is created. When Alice opens the app hours later, we do exactly one ZRANGE — no DB queries, no sorting, no computation. Just a read."

"Why does Fan-out on Read fail at Instagram scale?"

"Pull model is the intuitive first approach — user opens feed, you fetch the latest posts from everyone they follow, rank, return top 20. It works fine at small scale. At Instagram scale, the math kills you."

PULL MODEL – cost per feed request

Alice follows 500 people. She opens her feed.

```
FeedService.getFeed(userId=alice)
```

Step 1 Get Alice's followees

```
SELECT followee_id FROM followees_by_user
WHERE follower_id = 'alice' → 500 IDs returned (~5ms)
```

Step 2 For each followee → fetch their latest 20 posts

```
SELECT * FROM posts WHERE user_id = ?
ORDER BY created_at DESC LIMIT 20
← 500 queries fired in parallel
← even with full parallelism: ~50-100ms just for I/O
```

← $500 \times 20 = 10,000$ posts loaded into memory

Step 3 Rank 10,000 posts in memory (~15ms)

Step 4 Return top 20

Total: 150–300ms for ONE user. Requirement: < 200ms ✗

Scale to 500M DAU at peak hour:

500M users × 500 DB queries/user = 250 BILLION queries

Cassandra cluster can handle: ~2M reads/sec

Pull model needs: ~69M reads/sec ($250B \div 3600s$)

Shortfall: 35×

You'd need 35× your current DB infrastructure.

Not a tuning problem – a fundamental architecture problem.

"Pre-computation shifts that work to write time. 100M posts created per day — each post triggers fan-out to its followers. That's a bounded, predictable workload. Reads become single Redis lookups. The DB is shielded from feed traffic entirely."

"How does the score work — walk me through `ZADD feed:{userId} score postId`."

"The score is just a float that captures 'how relevant is this post right now?' Higher score = shown higher in the feed. It decays over time automatically."

SCORE FORMULA

$$\text{score} = \frac{(\text{likes} \times 1) + (\text{comments} \times 3) + (\text{shares} \times 5)}{(\text{hours_since_posted} + 2)^{1.5}}$$

NUMERATOR = engagement weight

likes ×1 low signal (one tap)

comments ×3 medium (person typed words)

shares ×5 high signal (person broadcast it to their followers)

DENOMINATOR = gravity

(hours + 2)^{1.5} grows fast:

hour 0 → 2^{1.5} = 2.8 tiny penalty

hour 6 → 8^{1.5} = 22.6 medium penalty

hour 24 → 26^{1.5} = 133 heavy penalty

hour 72 → 74^{1.5} = 637 buried

WORKED EXAMPLE – two posts competing for Alice's top slot:

Post A: 1 hour old, 200 likes, 20 comments, 3 shares

numerator = $200 + (20 \times 3) + (3 \times 5) = 275$

denominator = $(1+2)^{1.5} = 5.2$

score = $275 / 5.2 = 52.9$

Post B: 24 hours old, 5000 likes, 200 comments, 50 shares

numerator = $5000 + (200 \times 3) + (50 \times 5) = 6250$

denominator = $(24+2)^{1.5} = 132.6$

score = $6250 / 132.6 = 47.1$

Post A (52.9) ranks above Post B (47.1)

Even though Post B has 25× more likes – freshness wins.

GRAVITY DECAY VISUAL (post with 500 engagement points):

hour 0	$500/2.8 = 178.6$	
hour 6	$500/22.6 = 22.1$	
hour 24	$500/133 = 3.8$	
hour 72	$500/637 = 0.8$	
hour 168	$500/3330 = 0.15$	· (7 days – gone)

"How it flows into Redis:"

Fan-out Worker processes "post:bob:1234" (1hr old, 200L 20C 3S):

score = 52.9

```
Redis pipeline {
  ZADD feed:alice 52.9 "post:bob:1234" ← score + member
  ZADD feed:carol 52.9 "post:bob:1234"
  ZADD feed:dave 52.9 "post:bob:1234"
  ...× all of bob's followers
}
```

Alice's sorted set after ZADD:

99.1	post:other:9999	← some older viral post
52.9	post:bob:1234	← just inserted here
47.1	post:older:5678	
...		

Trade-off I'd flag: score is computed once at fan-out time.

As likes accumulate, the score in existing feeds becomes stale.

Fix: background re-scorer runs ZADD on trending posts every few minutes.

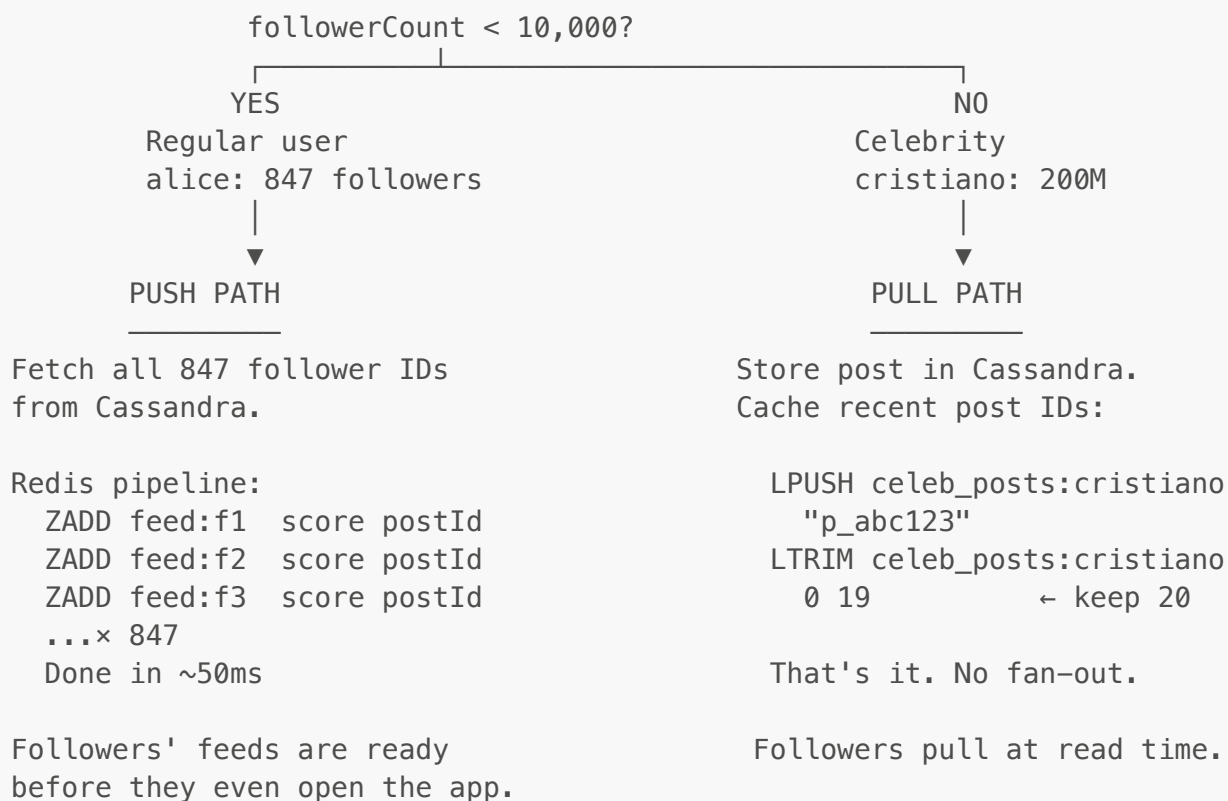
But I'd call that out as a known limitation, not solve it upfront.

"Walk me through push vs pull — why hybrid? Why 10K threshold?"

"Push-only breaks on celebrities. Pull-only breaks on read latency. The hybrid gives you fast reads AND avoids the celebrity write storm. Let me show the decision logic."

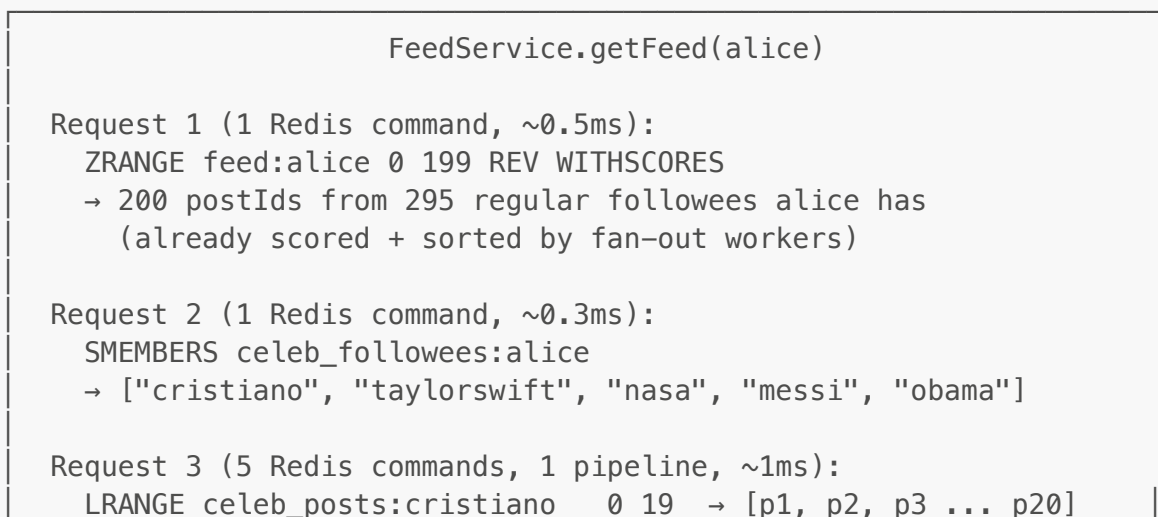
HYBRID FAN-OUT – decision at write time

POST_CREATED event: { postId, userId, followerCount }



"Why 10K? It's a tunable constant. The math: at 10K followers, fan-out takes about 50ms — well within acceptable async time. At 1M followers, it starts taking seconds. At 200M, it takes minutes. The threshold is where the fan-out time becomes unacceptably long for eventual consistency guarantees. Instagram reportedly uses something around this range."

AT READ TIME – hybrid merge (Alice follows 295 regular + 5 celebrities)



```

LRANGE celeb_posts:taylorswift 0 19 → [p1, p2 ... p20]
LRANGE celeb_posts:nasa         0 19 → [p1 ... p10]
LRANGE celeb_posts:messi        0 19 → [p1 ... p20]
LRANGE celeb_posts:obama        0 19 → [p1 ... p15]

```

In-memory merge (~0.5ms):

```

candidates = 200 regular posts + 85 celebrity posts = 285 total
filter: remove from unfollowed/blocked users

```

MGET metadata for 285 candidates (1 pipeline, ~5ms):

```

HGETALL post:p1, post:p2, ... post:p285
cache miss? → Cassandra read, backfill Redis

```

Re-rank 285 → top 20 → return (~8ms)

Total: ~35ms  (well under 200ms SLA)

"Walk me through feed ranking — the score formula and two-level approach."

"Chronological ordering sounds simple but kills engagement. If Alice follows 500 people, her best friend's post gets buried under 200 other posts made five minutes later. Ranking surfaces what's most relevant to Alice specifically.

The formula I'd use is the same gravity model as Hacker News and Reddit:"

SCORE FORMULA (already covered above, summarized here for flow)

score = engagement_weight / time_gravity

engagement_weight = likes×1 + comments×3 + shares×5

time_gravity = (hours_since_posted + 2)^1.5

Old posts sink. Engaging posts rise. Fresh engaging posts dominate.

"The two-level ranking is important to call out explicitly because it's how you make ML feasible in real time."

TWO-LEVEL RANKING PIPELINE

LEVEL 1: Candidate Generation (pre-computed, offline)

Redis sorted set: feed:alice (up to 500 entries)

Written by fan-out worker using engagement_gravity score.
Same score for all users – not yet personalized.

At read time:
 ZRANGE feed:alice 0 199 REV → 200 candidates in 0.5ms
 Add ~85 celebrity posts → 285 total candidates

| 285 candidates



LEVEL 2: Personalized Re-ranking (online, at request time)

For each of the 285 candidates:

affinity_score = HGET affinity:alice {postId}
 → 0.0 to 1.0 based on Alice's interaction history
 → Alice likes Bob's posts 90% of the time → affinity=0.9
 → Alice rarely engages with Carol → affinity=0.1

freshness_flag = GETBIT seen:alice {postId}
 → 0 if Alice has never seen this post (boost it)
 → 1 if already served (penalty or skip)

final_score = base_score × affinity_score × freshness_factor

Sort 285 posts by final_score → take top 20

Mark top 20 as seen: SETBIT seen:alice {postId} 1

| top 20



Return to Alice

CONCRETE EXAMPLE:

post:carol:5678 base=87.3, affinity(alice→carol)=0.1
 final = 87.3 × 0.1 = 8.7 ← sinks despite high base score

post:bob:1234 base=52.9, affinity(alice→bob)=0.9
 final = 52.9 × 0.9 = 47.6 ← rises because alice loves bob

Bob's post (lower base) beats Carol's (higher base)
 because Alice interacts with Bob far more.

"Why two levels? You can't run ML on thousands of candidates per request at 500M DAU — too expensive. Level 1 is cheap: one Redis ZRANGE. Level 2 is more intelligent but runs on only 285 posts. At that size, the re-ranking takes maybe 8ms. Same pattern used by YouTube, TikTok, Twitter. Candidate generation for recall, ranking for precision."

FULL PIPELINE LATENCY BREAKDOWN

ZRANGE pre-computed feed (200 posts):	~0.5ms
SMEMBERS celeb list:	~0.3ms
LRANGE × 5 celebrity caches (pipeline):	~1.0ms
MGET post metadata × 285 (pipeline):	~5.0ms

Affinity lookups × 285 (HGET pipeline):	~3.0ms
Re-rank 285 in memory:	~8.0ms
Serialize + network:	~20.0ms
Total:	~37.8ms 
(5× margin against 200ms SLA)	

A. Why Cassandra — Not MySQL / PostgreSQL?

The Problem

You need to store 1 trillion follow relationships (who follows whom) and 10 billion posts. MySQL is a single server by default — it cannot physically hold that much data.

What Happens Without Cassandra

Option A: One big MySQL server

1 trillion rows × 16 bytes = 16 TB

One MySQL server max = ~2–4 TB usable

→ Data doesn't fit. Impossible.

Option B: Manually shard MySQL across 10 servers

You split the data: server-1 holds users A–C, server-2 holds D–F, etc.

Now queries that cross shards are nightmare:

"Give me all followers of user Bob"

→ Bob might be on shard-3 but his followers are on shards 1, 2, 5, 7, 9

→ You query all shards → merge results → slow + complex code

→ SQL JOINS across servers = impossible

Also: if one shard goes down → 10% of your data is gone

Why Cassandra

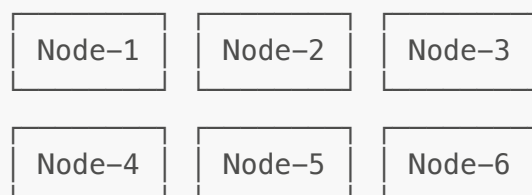
Cassandra was designed FROM THE START to run on 100+ servers simultaneously.

MYSQL (traditional):



Problem: too big for 1 server

CASSANDRA:



Each node holds 1/6 of the data.
Add more nodes = more storage.
Every piece of data has 3 copies
(on 3 different nodes) = safe.

Key Cassandra Properties

Property	What it means
Partition key	You choose HOW data is spread across nodes. <code>followee_id</code> as partition key = all followers of user X land on ONE node
No joins	You design tables specifically for your query. Two follow tables = two lookup directions
Linear scaling	Need more capacity? Add nodes. No downtime, no resharding
Tunable consistency	Can choose: write to 1 node (fast) or write to 3 nodes (safe)
No single point of failure	Every row exists on 3 nodes. One dies → 2 still serve it

What MySQL IS Good For in This System

User profiles, billing, auth → MySQL/PostgreSQL (ACID needed)

- These have LOW write volume
- Need strong consistency (you can't have duplicate usernames)
- Data is small (millions of rows, not trillions)

Posts, follow graph, comments → Cassandra

- Massive scale (trillions of rows)
- Eventual consistency is acceptable
- Query patterns are simple and known upfront (no ad-hoc joins)

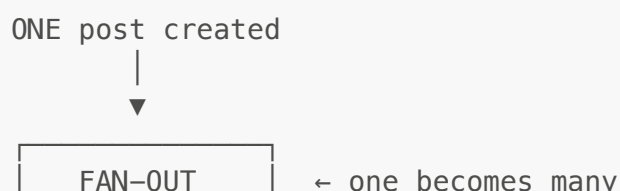
Interview One-Liner

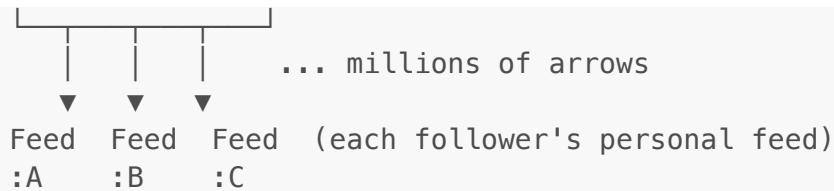
"MySQL can't horizontally scale to 1 trillion rows without painful manual sharding. Cassandra partitions data automatically, scales linearly by adding nodes, and is designed for the denormalized, known-query-pattern access we have here."

B. Fan-out — What Does It Mean?

Simple Real-World Analogy

Imagine you're a TV station. You film ONE show (create one post). You need to BROADCAST it to 10 million viewers. One signal → spreads out to many → that spreading is called **fan-out**.





In our system:

- Cristiano Ronaldo creates ONE post
- 200 million followers each need to see it in their personal feed
- Spreading that one post to 200M feeds = **fan-out**

Fan-out direction:

- **Fan-out on Write** (push): Do the spreading WHEN the post is created
- **Fan-out on Read** (pull): Do the spreading WHEN a follower opens the app

C. Why Do We Need a Fan-out Worker?

The Problem

When someone creates a post, you can't spread it to millions of feeds *synchronously* (blocking).

What Happens Without a Fan-out Worker

Cristiano posts → Post Service tries to do EVERYTHING:

1. Save post to DB
2. Find all 200M followers
3. Write to all 200M Redis caches
4. THEN return "post saved" to Cristiano's app

Step 2-3 takes MINUTES.

Cristiano's app spins forever. ❌

Why We Need It — Async Separation

WITHOUT Fan-out Worker:

POST REQUEST 1. Save to DB 2. Fan out to 200M 3. Return response	← takes minutes, user waits
---	-----------------------------

WITH Fan-out Worker (async):

POST REQUEST

- | | | |
|-----------------------------|---|----------------|
| 1. Save to DB | → | ✓ done in 10ms |
| 2. Publish Kafka event | → | ✓ done in 5ms |
| 3. Return "posted!" to user | → | ✓ instant |

↓ Kafka event (async, non-blocking)

Fan-out Worker (background, takes as long as needed) - Reads follower list from Cassandra - Writes postId to each follower's Redis feed cache - Does this in parallel across many workers - User sees post gradually appearing in followers' feeds within seconds (eventual consistency)
--

Execution Flow

```

t=0ms   Cristiano taps "Post"
t=10ms  Post saved to Cassandra
t=15ms  Kafka event "post-created" published
t=20ms  App returns ✓ to Cristiano – he sees "Posted!"

t=100ms Fan-out Worker picks up Kafka event
t=100ms Fetches follower IDs from Cassandra (millions of rows)
t=200ms Batches followers into chunks of 1,000
t=200ms Spawns 100 parallel workers, each handles 2M followers
t=5s    All 200M Redis caches updated
t=5s    All followers now see the post in their feed
  
```

Interview One-Liner

"The Fan-out Worker decouples post creation from feed propagation. Post creation returns immediately (< 20ms). The worker handles slow fan-out asynchronously via Kafka. This is the classic async write-behind pattern."

D. Pre-computed Sorted Sets / ZADD / ZRANGE — What They Are

What is a Redis Sorted Set? (Simple)

Think of it as a **leaderboard** where:

- Each entry = one post
- Each entry has a score (a number) + a member (the postId)
- The leaderboard is always sorted from highest score to lowest

Alice's feed sorted set (feed:alice):

Score	Member (postId)	
99.1	post-from-bob-123	← top of feed (highest score)
87.3	post-from-carol-456	
76.8	post-from-dave-789	
...		
(up to 500 entries)		

What ZADD Does

ZADD = **Z**sorted set **ADD** — "Add this postId with this score into the sorted set"

```
ZADD feed:alice 99.1 "post-from-bob-123"
```



If postId already exists → it **UPDATES** the score (no duplicates ever).

What ZRANGE Does

ZRANGE = **Z**sorted set **RANGE** — "Give me posts from position X to position Y"

```
ZRANGE feed:alice 0 19 → returns the top 20 postIds (index 0 to 19)
```

Result: ["post-123", "post-456", "post-789", ...]

This is $O(\log N)$ — even with 500 posts in the set, it's instant.

Why "Pre-computed"?

The work is done **AHEAD OF TIME** (when the post is created), not when Alice opens the app.

```
When Bob posts (Fan-out Worker runs):
ZADD feed:alice 87.3 "bobs-new-post"
ZADD feed:carol 87.3 "bobs-new-post"
ZADD feed:dave 87.3 "bobs-new-post"
... (for all of Bob's followers)
← all feeds updated immediately, work done NOW
```

When Alice opens app later:

```
ZRANGE feed:alice 0 19 → instant result, no computation needed!  
← all work was already done, just reading the result
```

Full Picture

WITHOUT pre-computation (naive pull):

Alice opens app

- query DB for all 500 followees' posts (500 queries!)
- merge 10,000 posts
- rank them
- return top 20

Total time: 500-1000ms ❌ (too slow)

WITH pre-computation (Redis sorted set):

Alice opens app

- ZRANGE feed:alice 0 19 (ONE Redis command)

Total time: < 1ms ✅

E. The Pull Model Problem — Why 500 DB Queries Kills You

The Problem

Fan-out on Read = compute Alice's feed WHEN she opens the app.

Step by Step — What Happens

Alice follows 500 people.
She opens her feed.

System must:

Step 1: Who does Alice follow?

DB query: `SELECT followee_id FROM follows WHERE follower=alice`

Result: [bob, carol, dave, eve, ... 500 users]

Time: ~5ms

Step 2: For each of 500 followees, get their latest 20 posts

`SELECT * FROM posts WHERE user_id = bob ORDER BY time`

`SELECT * FROM posts WHERE user_id = carol ORDER BY time`

`SELECT * FROM posts WHERE user_id = dave ORDER BY time`

... × 500 queries

Time: $500 \times \sim 2\text{ms} = 1000\text{ms} = 1 \text{ SECOND}$ minimum ❌
(even with parallelism: still 200–500ms)

Step 3: Merge results

500×20 posts = 10,000 posts in memory

Sort all 10,000 by score

Time: $\sim 10\text{ms}$ in memory (fast, but wasted work)

Step 4: Return top 20 to Alice

Total: 500ms–1s just for ONE user. Requirement: < 200ms. FAIL.

The Math That Kills You

500M users open app during peak hour

Each user triggers 500 DB queries

= $500\text{M} \times 500 = 250 \text{ BILLION}$ DB reads per hour

= 69 MILLION DB reads per second

Instagram's entire Cassandra cluster handles $\sim 1\text{--}2\text{M}$ reads/sec.

This approach needs 35× more DB capacity than they have.

Why It "Works" at Small Scale But Fails at Instagram Scale

10 users $\times$ 500 queries = 5,000 queries → any DB handles this fine

1,000 users $\times$ 500 queries = 500K queries → still manageable

1M users $\times$ 500 queries = 500M queries → starts to hurt

500M users $\times$ 500 queries = 250B queries → system dies

This is why the pre-computation approach exists: shift work from read time to write time.

F. The Score — How It Works and Why It Helps

The Formula (Simple)

$$\text{score} = \frac{(\text{likes} \times 1) + (\text{comments} \times 3) + (\text{shares} \times 5)}{(\text{hours_since_posted} + 2)^{1.5}}$$

Numerator = engagement weight

- Likes are worth 1 point (easy to click)
- Comments are worth 3 points (took effort to type)
- Shares are worth 5 points (person loved it so much they showed their friends)

Denominator = gravity (time decay)

- The longer ago it was posted, the bigger the denominator, the smaller the score
- +2 prevents division by zero for brand-new posts
- $^{1.5}$ means decay accelerates fast

Worked Example

Post A: posted 1 hour ago, 200 likes, 10 comments, 2 shares

Numerator: $(200 \times 1) + (10 \times 3) + (2 \times 5) = 200 + 30 + 10 = 240$

Denominator: $(1 + 2)^{1.5} = 3^{1.5} = 5.2$

Score A = $240 / 5.2 = 46.2$

Post B: posted 12 hours ago, 5000 likes, 300 comments, 100 shares

Numerator: $(5000 \times 1) + (300 \times 3) + (100 \times 5) = 5000 + 900 + 500 = 6400$

Denominator: $(12 + 2)^{1.5} = 14^{1.5} = 52.4$

Score B = $6400 / 52.4 = 122.1$

Post C: posted 72 hours ago, 100K likes, 20K comments, 5K shares

Numerator: $100000 + 60000 + 25000 = 185000$

Denominator: $(72 + 2)^{1.5} = 74^{1.5} = 637$

Score C = $185000 / 637 = 290$

Ranking: Post C > Post B > Post A

Even though Post A is newest, Post C's massive engagement wins.

How This Score Gets Into Redis

Fan-out Worker calculates score when post is created:

```
score = compute_score(likes=200, comments=10, shares=2, hours=1)
      = 46.2
```

```
ZADD feed:alice 46.2 "post-from-bob-123"
```

```
ZADD feed:carol 46.2 "post-from-bob-123"
```

```
ZADD feed:dave 46.2 "post-from-bob-123"
```

When Alice opens her feed, posts are returned in descending score order. Highest score = most relevant = shown first.

Score Updates Over Time

As a post gets more likes, its score should increase. As hours pass, its score should decrease.

Solution: Notification Worker periodically updates scores:

Every 5 minutes for posts < 1 hour old

Every 30 minutes for posts 1–24 hours old

No update for posts > 24 hours old (score frozen, will decay naturally)

```
Re-run score formula with latest like/comment counts:  
ZADD feed:alice <new_score> "post-bob-123" ← ZADD updates existing  
score
```

G. Push vs Pull Hybrid — Simple Analogy

Why Push-Only Fails

```
Cristiano has 200M followers. He posts.  
Fan-out Worker: "I must write to 200M Redis caches!"  
200M × ZADD = 200M Redis write commands  
At 100K writes/sec → takes 33 MINUTES  
Followers see post 33 minutes late ❌
```

Why Pull-Only Fails

Already explained above — 500 DB queries per user, too slow.

The Elegant Hybrid Solution

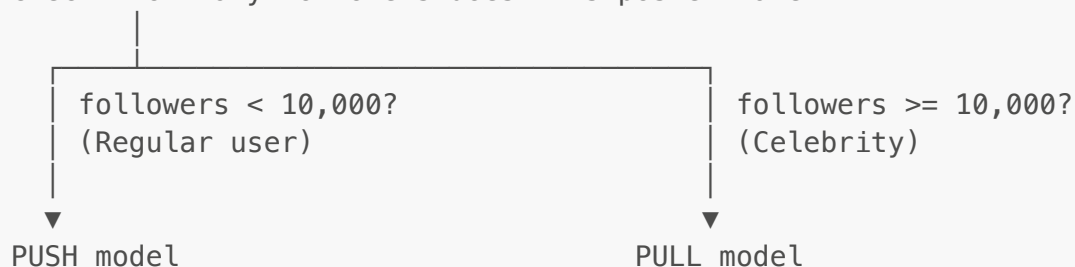
Think of it this way:

```
Your local cafe owner posts on Instagram:  
Has 847 followers → like a neighborhood newsletter  
Fine to individually drop a copy in 847 mailboxes (PUSH)  
Only takes seconds  
  
Cristiano Ronaldo posts:  
Has 200M followers → like a major TV broadcast  
Can't knock on 200M doors individually (too slow)  
Instead: broadcast once, viewers tune in themselves (PULL)
```

Implementation Decision

When Fan-out Worker receives "post-created" event:

Check: how many followers does this poster have?



For each follower:

```
ZADD feed:{followerId}  
    score postId
```

When follower opens app:

```
ZRANGE feed:userId 0 19  
= instant result ✅
```

Just store post in DB.

Mark poster as celebrity.
No fan-out at all!

When follower opens app:

Fetch from celebrity's
post cache on-demand ✅

Why This Works

Regular users:

Max 9,999 followers → 9,999 ZADD commands → done in ~50ms
Feed is pre-built → reads are instant

Celebrities:

Their posts are HOT (millions of people read them)
Hot posts are cached in Redis anyway
5-10 celebrity lookups at read time = 5-10 Redis commands = fast
No 200M write storm

H. Hybrid Merge Feed Read Flow — Step by Step

Alice follows 300 regular users and 3 celebrities (Cristiano, Taylor Swift, NASA).

Alice opens her feed...

STEP 1: Get Alice's pre-computed feed

```
ZRANGE feed:alice 0 199 WITHSCORES REV
```

→ returns 200 post IDs from regular users she follows
→ ultra fast (1 Redis command)

feed:alice sorted set:

```
[post-bob-123, post-carol-456, post-dave-789, ... x200]
```



STEP 2: Find out which celebrities Alice follows

```
SMEMBERS celeb_followees:alice
```

→ returns ["cristiano", "taylorswift", "nasa"]
→ 1 Redis command



STEP 3: Fetch recent posts from each celebrity

```
GET celeb_posts:cristiano → [post-C1, post-C2, post-C3]
GET celeb_posts:taylorswift → [post-T1, post-T2]
GET celeb_posts:nasa → [post-N1]
→ 3 Redis commands (one per celebrity)
→ Note: these keys are HOT (millions of reads/sec), fast
```

**STEP 4: Merge all candidates**

```
200 posts (pre-computed)
+ 6 posts (from celebrities)
= 206 total candidates
```

Remove any post from someone Alice unfollowed or blocked

**STEP 5: Fetch post metadata for all 206 candidates**

```
MGET post:C1 post:C2 post:T1 post:bob123 ...
(Redis multi-get, 1 round trip, fast)
Cache miss? → read from Cassandra, backfill Redis
```

**STEP 6: Re-rank the 206 candidates**

```
For each post, compute final score using affinity:
  final_score = base_score × (does Alice usually like Bob?)
```

```
Sort 206 posts by final_score
Take top 20
```

**STEP 7: Return to Alice**

```
{ posts: [top 20], nextCursor: "post-id-of-20th" }
Total time: < 50ms (mostly Redis, tiny ML step)
```

Why This Meets the 200ms Requirement

Step 1 (ZRANGE):	~1ms
Step 2 (SMEMBERS):	~0.5ms
Step 3 (3 GET commands):	~1.5ms
Step 4 (merge in-memory):	~0.5ms
Step 5 (MGET 206 keys):	~5ms
Step 6 (re-rank 206 posts):	~10ms
Step 7 (serialize response):	~5ms
Network overhead:	~20ms
Total:	~44ms  (well under 200ms)

I. Feed Ranking — Why and How

The Problem with Chronological Ordering

Alice follows 500 people.
All 500 post at roughly the same time (morning commute).

Chronological order:

- 8:03am – Random influencer posts a meme
- 8:02am – Another person posts lunch photo
- 8:01am – Alice's best friend posts important life news
- 8:00am – Alice's mom posts a family photo

Alice sees the meme first. She might scroll past her best friend's news.
Chronological order doesn't match "what Alice cares about most."

What Happens Without Ranking

- Users see random posts in time order
- Engagement (likes/comments) drops
- Users open app less often
- Business impact: fewer ads shown, less revenue
- This is why every major social platform has ranking

Why We Need Ranking

Show Alice the posts she's MOST LIKELY to engage with, at the top. This increases time-on-app and engagement.

The Score Formula — Broken Down Simply

$$\text{score} = \frac{(\text{likes} \times 1) + (\text{comments} \times 3) + (\text{shares} \times 5)}{(\text{hours_since_posted} + 2)^{1.5}}$$

NUMERATOR tells us: how engaging is this post?

likes	× 1	= clicks (easy)
comments	× 3	= typed text (effort)
shares	× 5	= rebroadcast (loved!)

DENOMINATOR tells us: how old is this post?

hours=0	→ denominator = 2 ^{1.5} = 2.8	(barely)
hours=6	→ denominator = 8 ^{1.5} = 22.6	(medium)
hours=24	→ denominator = 26 ^{1.5} = 133	(heavy)
hours=72	→ denominator = 74 ^{1.5} = 637	(buried)
→ Old posts SINK even if they had lots of likes		

Gravity in Action (Visual)

Post score over time (starts at 100 points of engagement):

Hour 0:	score = 100 / 2.8 = 35.7	
Hour 6:	score = 100 / 22.6 = 4.4	
Hour 12:	score = 100 / 56.6 = 1.8	
Hour 24:	score = 100 / 133 = 0.75	
Hour 48:	score = 100 / 377 = 0.27	
Hour 72:	score = 100 / 637 = 0.16	

- Post naturally sinks to the bottom over 2-3 days
- A NEW post with 20 likes will outrank a 2-day-old post with 10K likes
- This ensures feed always feels "fresh"

Two-Level Ranking — Why

Problem:

Alice's feed cache has 200 candidates.

Can't run a full ML model on all 200 in real time (too slow + expensive).

But basic score isn't personalized (same score for all users).

Solution:

LEVEL 1: Candidate Generation (done by Fan-out Worker)
--

```

Compute basic score for each post (formula above)
Store in Redis sorted set with this score
At read time: ZRANGE gets top 200 candidates cheaply

LEVEL 2: Re-ranking (done at read time, fast because <200 posts)

Apply personalization signals:
→ Does Alice frequently like Bob's posts? (affinity score)
→ Has Alice already seen this post? (freshness)
→ Is this Alice's first time seeing it? (novelty boost)

final_score = base_score × affinity_multiplier
Sort 200 by final_score → return top 20 to Alice

```

Why two levels?

ML on 200 posts is fast (< 10ms).

ML on 10,000 posts (all followees' posts) would be slow (> 1s).

Level 1 pre-filters with cheap math.

Level 2 personalizes the small shortlist. Best of both.



Interview One-Liner

"Two-level ranking: Redis pre-filters to 200 candidates using a cheap gravity score (same formula as HackerNews). ML re-ranks those 200 with personalization signals. This separates scale (Redis handles millions) from intelligence (ML handles a tiny shortlist)."

Good luck with your interview! The key differentiator is explaining the hybrid fan-out pattern and WHY it exists — the celebrity problem — before the interviewer asks about it.

Return to [System Design Index](#)